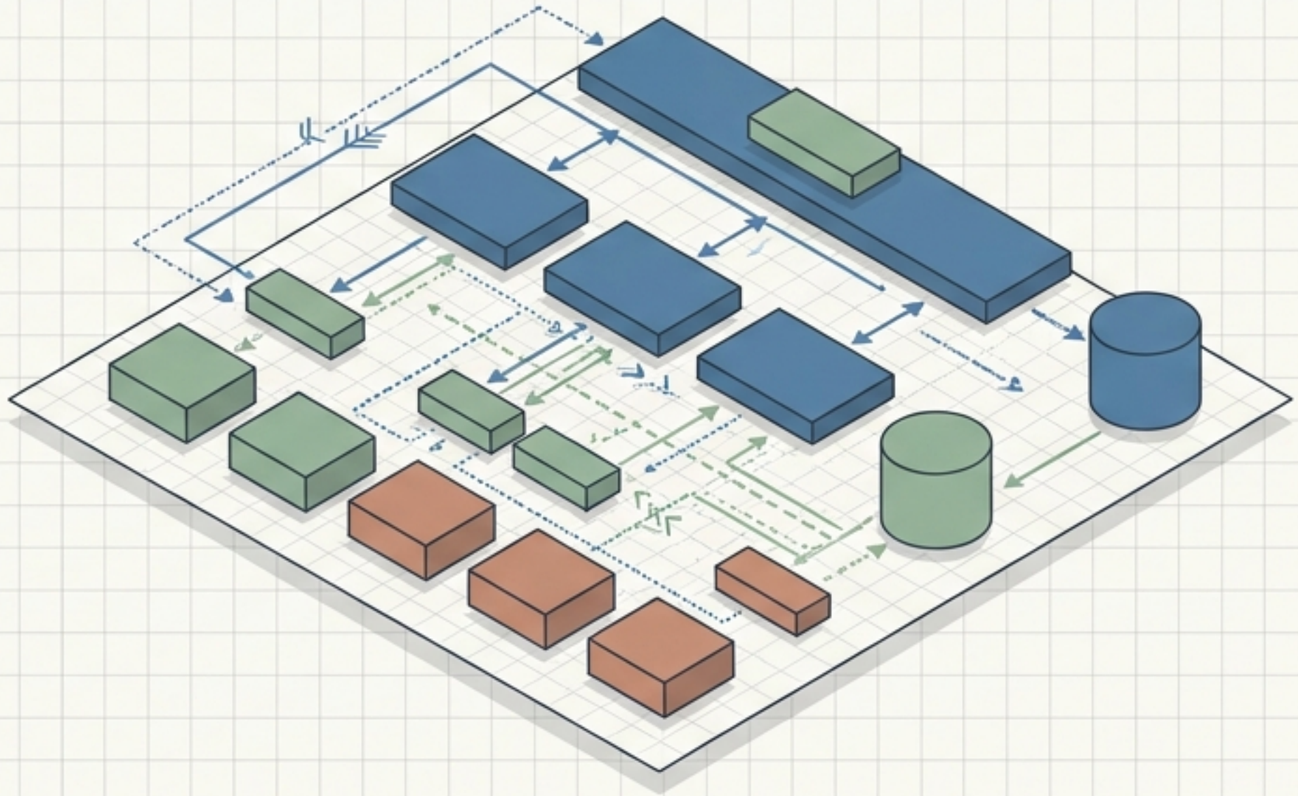
Hope (HireMe) System Architecture

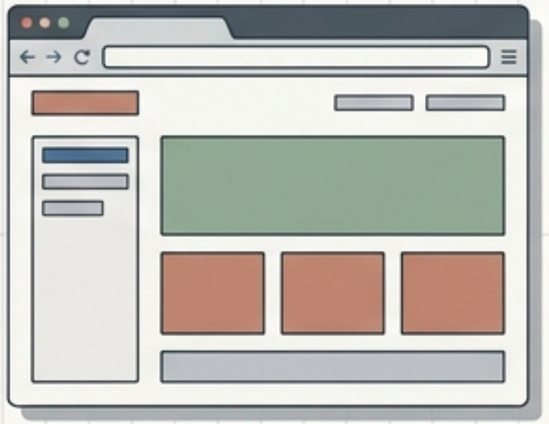
The Engineering Blueprint to Production

v1.0 MICROSERVICES EVENT-DRIVEN DDD



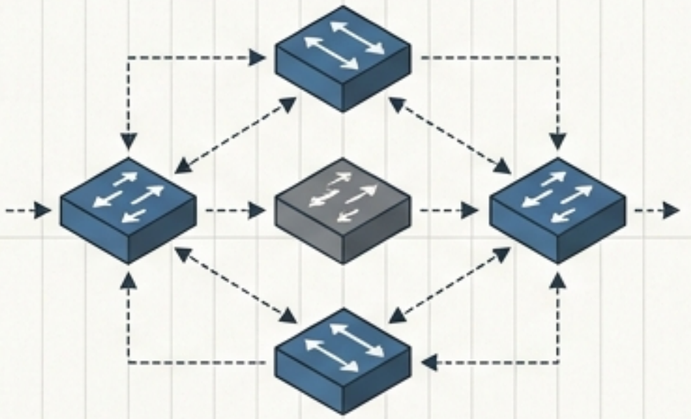
Architectural Philosophy: Separation of Concerns

Presentation Layer (Frontend)



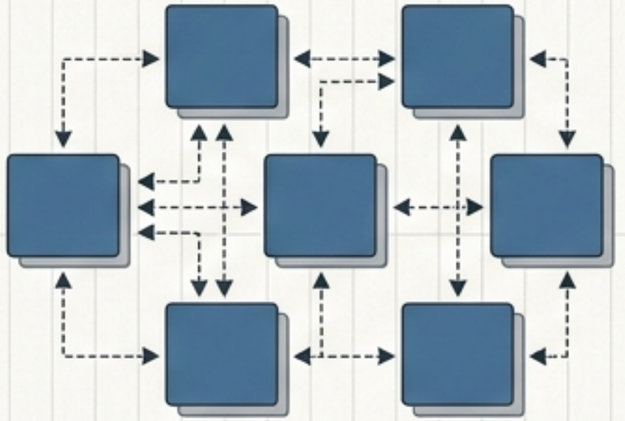
- **React 19** SPA built on **Atomic Design** principles.
- **TanStack Start** file-based routing mapping 33 seamless routes.
- Consumes backend exclusively via the internal **API Gateway**.

Infrastructure Layer (The Backbone)



- Cross-cutting operational concerns and internal routing.
- **Spring Cloud Gateway** for edge boundary.
- **Netflix Eureka** for service discovery.
- **Spring Cloud Config** for Git-backed centralized configuration.

Business Services Layer (Core Logic)



- Eight highly isolated, domain-aligned microservices.
- **Users, Companies, Jobs, Applications, Resumes, Preferences, AI, Notifications.**
- Each module owns a single, strictly bounded business capability.

The Modern Tech Stack

Backend Runtime

Java

21 + Spring Boot 4.0.6

Microservices Platform

Spring Cloud

2025.1.0 + Eureka Server

Frontend Framework

React

19 + TanStack Router 1.168.25

Relational Database

PostgreSQL

16 + Flyway Migration

Message Broker

Apache Kafka

4.1.1 (KRaft mode)

API Gateway

Spring Cloud Gateway

MVC 2025.1.0

Containerization

Google Jib

Maven Plugin 3.5.1

Frontend Build & SSR

Vite

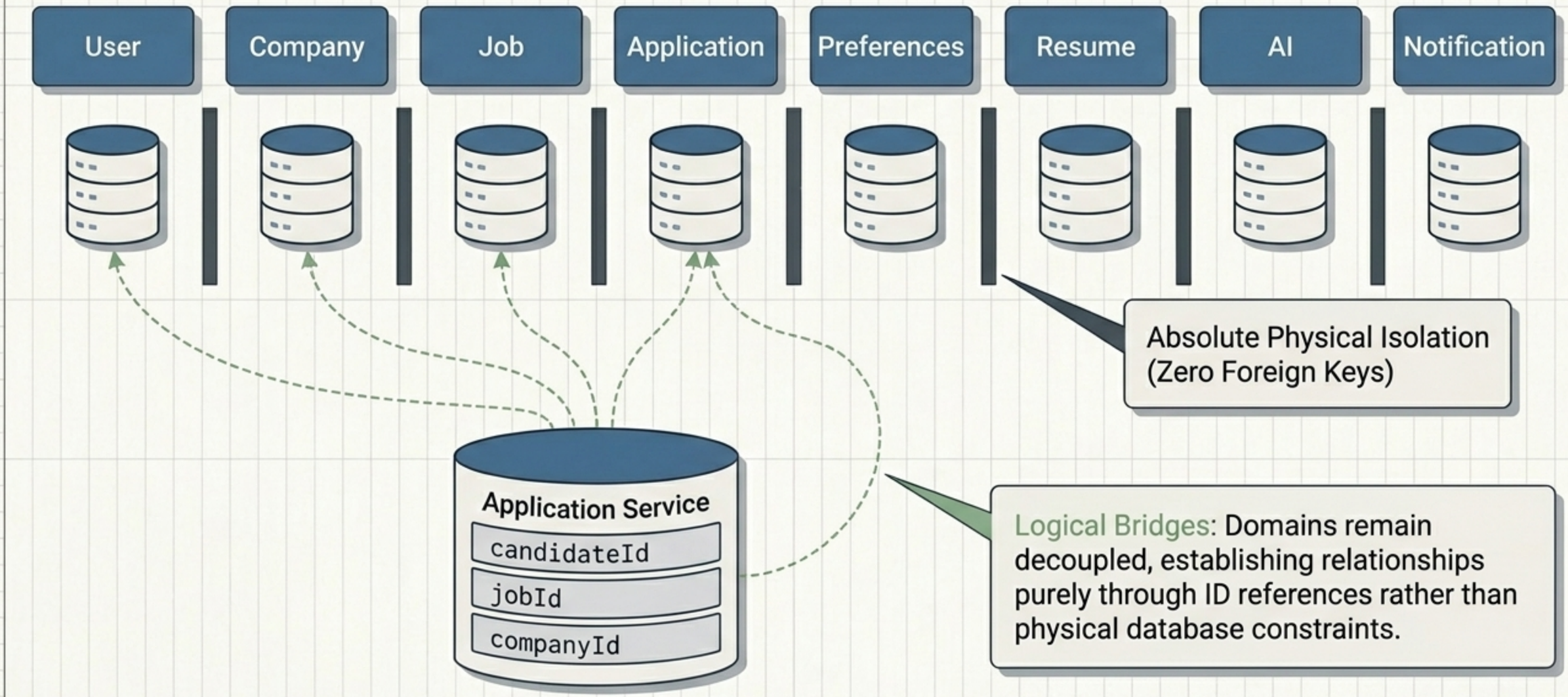
7.3.1 + Nitro 3.0.x beta

Artificial Intelligence

Google Gemini

gemini-2.5-flash-lite

Domain Isolation: Database-per-Service



Anatomy of a Microservice

Controller Layer
REST Endpoints mapped to frontend actions

Service Layer
Business logic (Interface/Implementation split for isolated testing)

Repository Layer
Spring Data JPA data access and queries

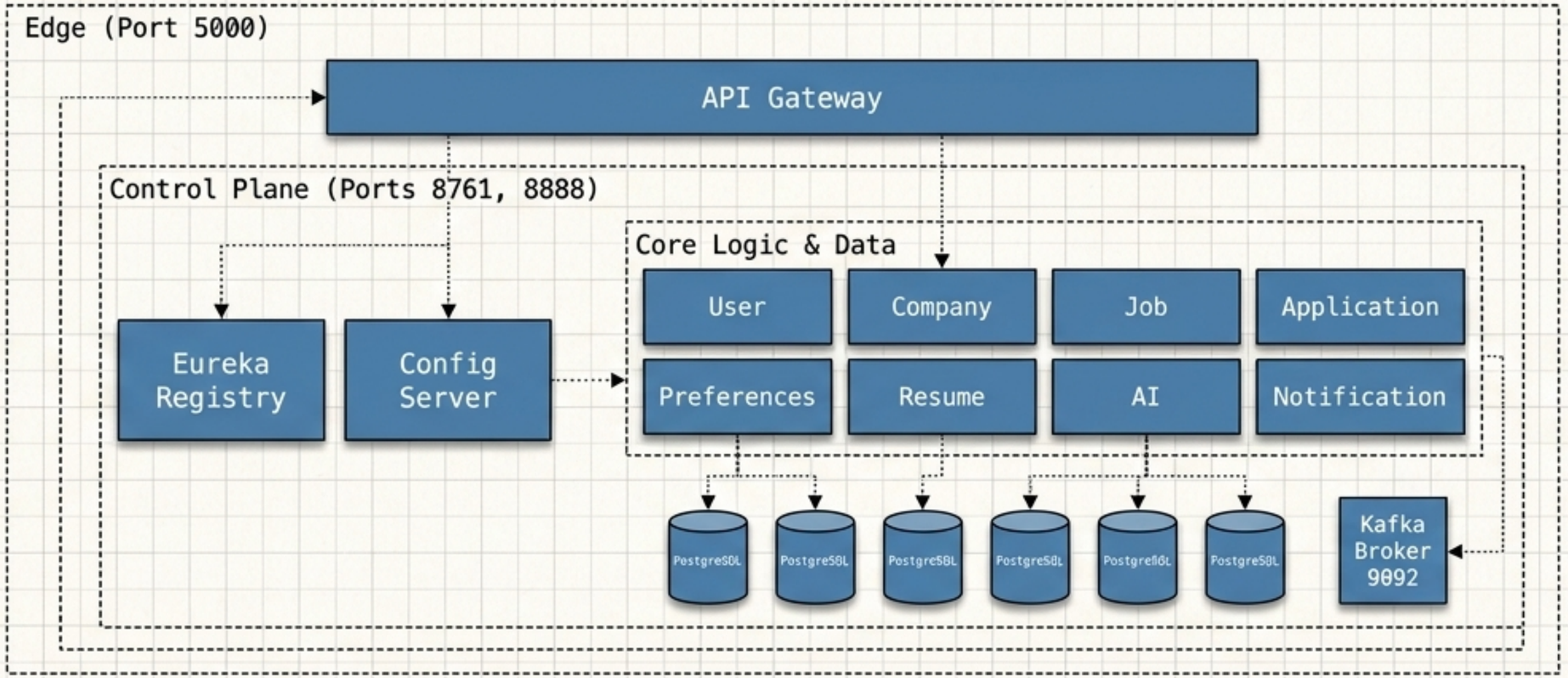
Entity Layer
Domain models and relational DB mapping

The Shared Kernel (common-lib module)

1. **Domain Enums:** ApplicationStatus, JobStatus, UserRole
2. **Shared Contracts:** ApiResponse, ErrorResponse
3. **Base Exceptions:** BusinessException, ResourceNotFoundException
4. **Domain Events:** ApplicationStatusChangedEvent

Ensures uniform API contracts and consistent domain vocabulary across all bounded contexts.

17-Container Macro Topology



Startup Dependency Sequence

Phase 1: Infrastructure Boot

Registry & Config pass /actuator/health checks (8 retries)

→

Phase 2: Data Persistence

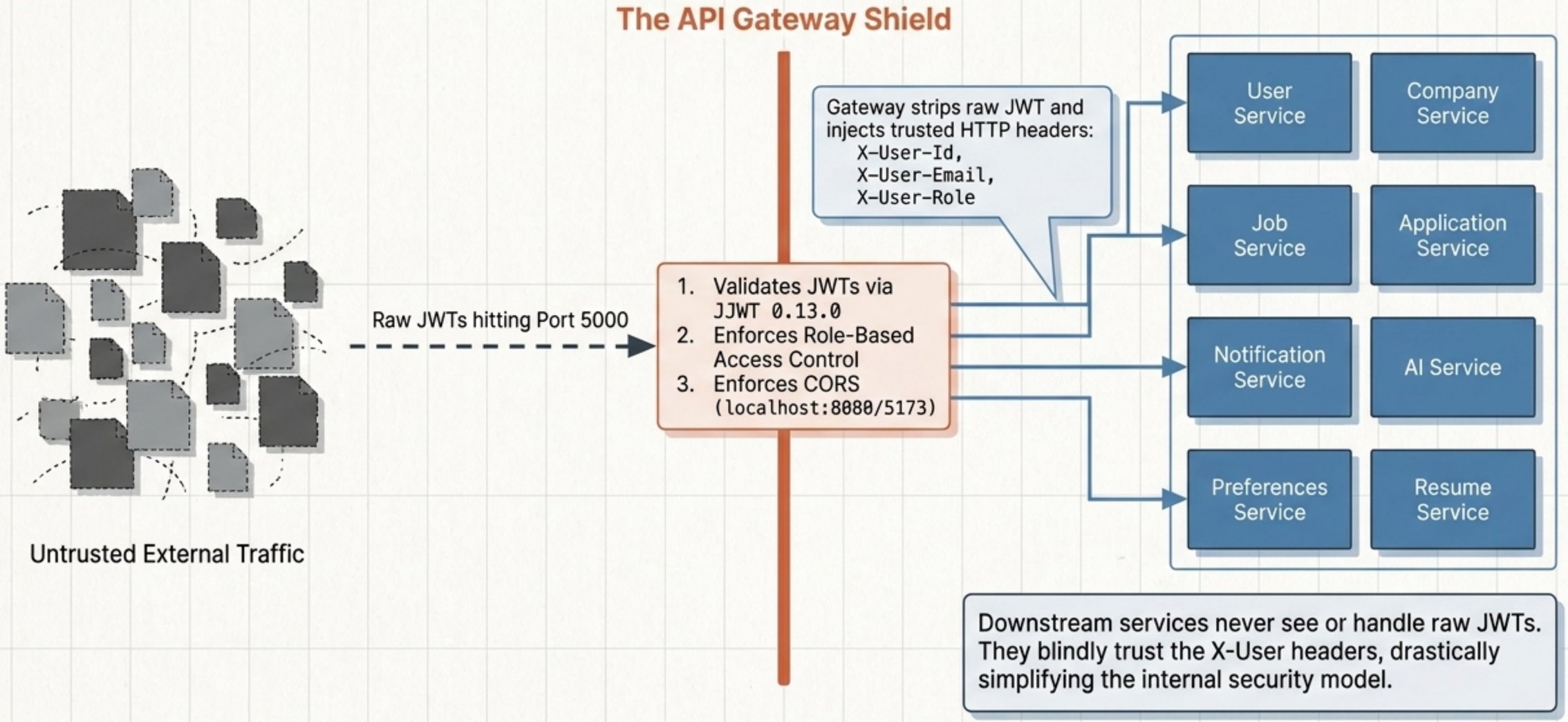
Databases boot & pass pg_isready checks (5 retries). Kafka initializes.

→

Phase 3: Business Logic

The 8 microservices boot only once infrastructure and databases are fully healthy.

Edge Security & The Trust Boundary



The Dual Communication Engine

	Synchronous (Query & Context)	Asynchronous (Workflows & Side-Effects)
Protocol & Tool	`REST / Spring Cloud OpenFeign`	Message Broker / `Apache Kafka (KRaft mode)`
Primary Purpose	Operations requiring immediate context and return data.	Side-effect-driven actions processed in the background.
Architectural Impact	Tighter coupling. Immediate failure if target service is down.	Ultimate decoupling. Producers fire and forget.
System Examples	AI Service gathering resume and job data via Feign clients.	Triggering email alerts when an application status changes.

Event-Driven Workflows: Kafka Decoupling

1. The Trigger:
User updates a candidate's application status.

2. Core Logic:
Application Service commits to Postgres DB.

3. The Decoupling Point:
Kafka Broker

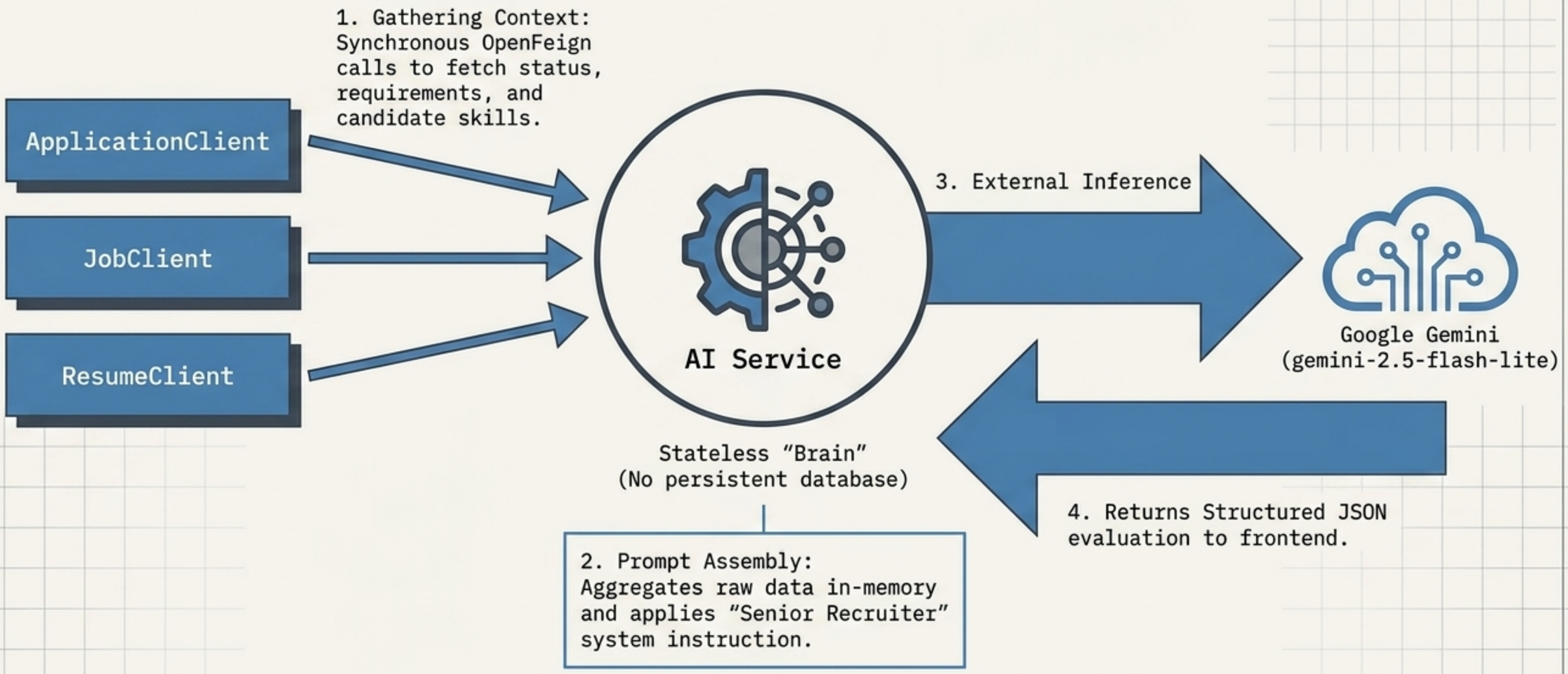
`application.status.changed`

Core business flow complete. Immediate success response sent to user.

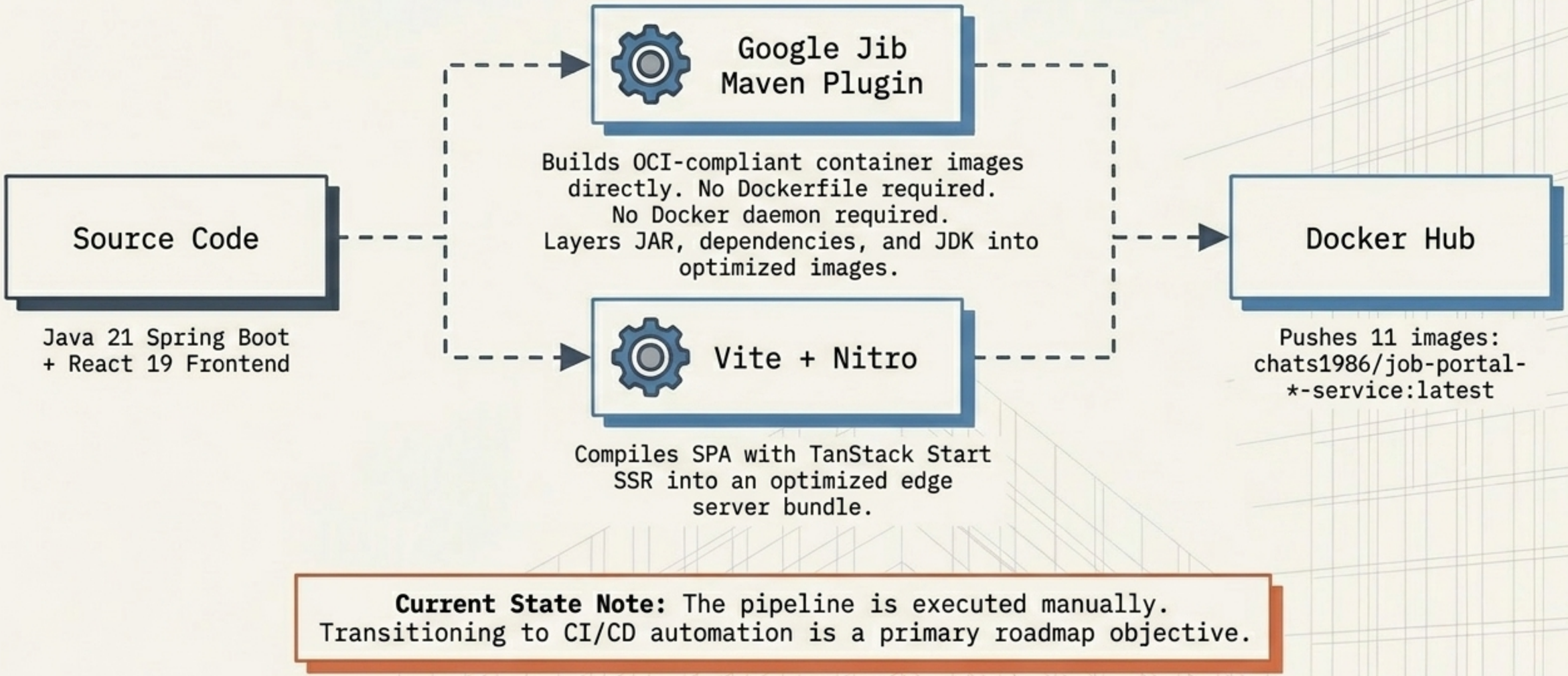
4. Consumption:
Notification Service picks up event via `@KafkaListener`.

5. Side-Effect:
Email sent. If email fails, core application data remains safely intact.

The Stateless AI Orchestrator



The Daemonless Deployment Pipeline



Current State Note: The pipeline is executed manually. Transitioning to CI/CD automation is a primary roadmap objective.

Scalability Matrix & Resource Realities

Scalability Strategy

Stateless Services (Business Core, Gateway)

Infinite horizontal scalability to N instances behind Eureka load balancer.

Infrastructure (Eureka, Kafka)

Currently single-node. Requires peer clustering for High Availability.

Databases (PostgreSQL)

Scale requires read-replicas and connection pooling (PgBouncer).

Current Resource Allocation

Memory Limit: 512 MB per container

Memory Reservation: 256 MB per container

CPU: 0.5 cores max per container

Kubernetes Readiness: System is architecturally primed for K8s. Containers are stateless with native /actuator/health probes ready for liveness/readiness checks.

Production Reality Check

1. Ephemeral Persistence

The Reality: No persistent Docker volumes are mapped for PostgreSQL or Kafka's KRaft log directory (/tmp/kraft-combined-logs).

The Risk: Absolute data loss upon container restart.

2. Hardcoded Secrets

The Reality: JWT secret keys, Gemini API keys, and SMTP credentials are in plaintext.

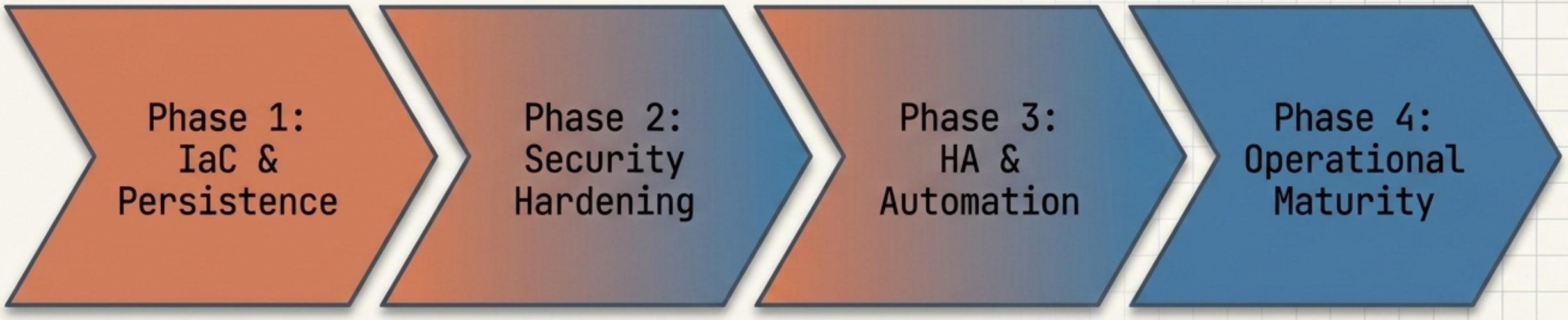
The Fix: Immediate implementation of HashiCorp Vault or Kubernetes Secrets manager.

3. Token Vulnerabilities

The Reality: Single shared JWT secret generates and validates tokens. No revocation or blocklist.

The Risk: Internal downstream services implicitly trust headers without mutual TLS verification.

The Enterprise Roadmap



- Create Terraform and K8s manifests.
- Mount persistent block storage for all 6 DBs and Kafka.

- Extract plaintext secrets to Vault.
- Upgrade to RS256 asymmetric keys.
- Implement mutual TLS between Gateway and internal services.

- Cluster Eureka and deploy 3-node Kafka.
- Implement GitHub Actions for CI/CD Jib builds.

- Standardize Flyway across all services.
- Add Prometheus + Grafana monitoring.
- Implement /v1/ API versioning at the Gateway.